

Assignment

Module Code: CT052-3-M-NLP

Module Name: Natural language Processing

Intake Code: APUMF2508DSBA(DE)

Programme: Masters in Data Science and Business Analytics.

Submitted By:

Student Name 1: Tashfia Choudhury (**TP Number:** TP091157).

Student Name 2: Mina Merati (**TP Number:** TP102564).

Student Name 3: Nida Noor Fathima (**TP Number:** TP100854).

Submitted To:

Module Lecturer: Raheem Mafas.

Table of Contents

Task 1 – Spelling Correction System 5

1. Introduction 5

1.1 Introduction to Candidate Techniques 5

1.1.1 Tokenization 6

1.1.2 Dictionary-Based Spell Checking 6

1.1.3. Minimum Edit Distance 7

1.1.4. Unigram and Bigram Language Models 7

1.1.5. GUI of Spell Checker 8

1.2 Summary 8

2. Edit Distance 9

2.1 Types of Edit Distance Method 9

2.2 Comparison between Edit Distance Types 11

3. Formulation and System Design 13

3.1 Dataset Selection and Corpus Construction 13

3.2 Overall System Architecture 14

3.3 Detection and Correction Strategy 14

3.4 Graphical User Interface Design 15

3.5 Functional Flowchart of the System 16

3.6 Summary 17

4. Implementation 18

4.1 Development Environment and Libraries 18

4.2 Text Processing and Tokenization 18

4.3 Corpus Construction and Dictionary Generation 19

4.4 Language Model Construction 19

4.5 Minimum Distance and Candidate Generation 20

4.6 Detection of Non-Word and Real-Word Mistakes 20

4.7 Graphical User Interface (GUI) 21

4.8 Efficiency and Optimization of Implementation 21

4.9 Summary 22

5. Results 23

5.1 Test Case 1: Detection of Non-Word Mistake 23
5.2 Test Case 2: Detection of Real-Word Mistake Using Context 24
5.3 Test Case 3: Managing Scientific Domain Vocabulary 25
5.4 Test Case 4: Detecting Mixed Mistake Types 26
6. Conclusion 28
7. References 29
Task 2 – Text Classification System (Sentiment Analysis) 30
1. Introduction 30
1.1 Introduction to Candidate Techniques 30
1.1.1 Text Preprocessing and Feature Extraction 30
1.1.2 Classical Machine Learning Classifiers 31
1.1.3 Deep Learning Approaches 31
1.1.4 Model Evaluation Metrics 31
1.2 Summary 32
2. Dataset and Exploratory Data Analysis 33
2.1 Dataset Selection and Justification 33
2.2 Exploratory Data Analysis 33
Class Balance 33
Review Length Distribution 34
Most Frequent Words per Class 34
Bigram Analysis 35
Word Clouds 35
2.3 Summary 35
3. Formulation and System Design 36
3.1 Preprocessing Pipeline 36
3.2 Overall System Architecture 36
3.3 Model Design and Selection Strategy 37
3.4 Summary 37
4. Implementation 38
4.1 Development Environment and Libraries 38
4.2 Data Loading and Cleaning 38

4.3 Feature Extraction (TF-IDF Vectorisation) 38
4.4 Model 1: Naive Bayes and Logistic Regression (Baseline) 39
4.5 Model 2: SVM and Random Forest (Tuned) 42
4.6 Model 3: Bidirectional LSTM (Deep Learning) 44
4.7 Efficiency and Optimisation of Implementation 46
4.8 Summary 46
5. Results 47
5.1 Overall Model Comparison 47
5.2 Hyperparameter Tuning Results 48
5.3 Discussion of Results 48
5.4 Model Deployment 49
6. Conclusion 53
7. References 54

Task 1 – Spelling Correction System

1. Introduction

The most essential task in Natural Language Processing (NLP) is Spelling Correction System which plays an important role in many kinds of text-processing applications like word processors, digital assistants, information retrieval programs and academic writing tools. The goal of the spelling correction system is to provide the best solution based on contextual data and linguistic similarities by identifying misspelled words. Automatic spelling correction is still a challenging issue even with advanced NLP techniques and algorithms. This happens because spelling errors occur in different ways and most of the time it requires more than simple dictionary matching.

Spelling errors are divided into two categories: 1) non-word errors and 2) real-word errors. The words which are not part of vocabulary are known as non-word errors (like qantum instead of quantum). By using dictionary lookup with similarity measures, it helps to detect this type of mistakes. On the other hand, real-word error means the words which are correctly spelled but used in an inappropriate context (like affect instead of effect). Here both words belong to dictionary entries so identifying these types of errors needs contextual analysis using probabilistic language models like n-gram models (Wang et al., 2021).

Research studies also highlighted additional problems as they contain specialized abbreviations, technical vocabulary and terminology. So, this study will use arXiv scientific corpus which contains more than 1,00,000 words to build a domain-based dictionary and probabilistic language model. By reviewing several candidate techniques, this study selected tokenization, dictionary-based spelling check, minimum edit distance, unigram and bigram language models. Lastly a graphical user interface (GUI) deployed to give access to user to input text, look through the corpus dictionary, detect spelling mistakes for both non-word and real-word spelling mistakes and get correction suggestions which will provide a correct solution for improving the quality of scientific writing.

1.1 Introduction to Candidate Techniques

There are several interconnected methods for spelling correction system that work combinedly to detect spelling mistakes and suggest appropriate corrections. Modern spelling checker systems combine many NLP methods to improve the contextual understanding, efficiency and accuracy of the system rather than relying on a single method. This study chose candidate techniques based on their efficiency in handling scientific text, compatibility with probabilistic language modeling and proven efficiency in earlier research.

This study's system developed using Tokenization to extract correct word units from user input, Minimum Edit Distance to produce candidate corrections for non-word mistakes and Unigram and Bigram Language Models trained on a scientific corpus to calculate probability of word and pair

of words. These probabilistic models made context-based ranking possible which helps to detect both non-word and real-word mistakes where the incorrect word belongs in the dictionary but used in inappropriate context. By combining all these methods, it develops a solid and practical system for detecting and correcting spelling mistakes in scientific writing.

1.1.1 Tokenization

Fundamental first pre-processing method of NLP is tokenization. This method divides the raw text into smaller units known as tokens which represent individual words, punctuations marks and numbers. The quality of tokenization directly affects the future NLP tasks perform because other processes like spell checker, information retrieval and language modeling rely on these generated tokens. According to Eisenstein (2019), effective tokenization can guarantee that text is represented in an organized and consistent way which helps NLP system to interpret input text accurately and helps to improve the performance of downstream tasks such as text classification and information retrieval.

It is mandatory to preprocess the input text before performing spelling check and correction. Because it helps to remove unnecessary variations that may decrease the accuracy of the correction step. This step includes converting characters to lowercase, removing punctuation, extra whitespace, special symbols and dividing the text into meaningful words. This process reduces noise in the corpus and ensures that identical words are uniformly translated no matter how they were originally formatted. Also, according to Dagan et al (2024) effective tokenization plays a vital part in generating vocabulary because it helps to determine how text is converted to consistent token. This influences model efficiency and downstream directly (Dagan et al., 2024).

Tokenization is used in this study by extracting only alphabetic words while ignoring punctuation, numbers and special characters from the raw input text using python's regular expression functions. Then extracted tokens are used to develop a domain-specific dictionary and used to make unigram and bigram models from the arXiv scientific corpus (Cassee et al., 2024). This study corpus contains scientific publications with technical vocabulary, that's why it needs an efficient tokenization method to maintain meaningful terms while eliminating irrelevant tokens.

1.1.2 Dictionary-Based Spell Checking

Then common technique dictionary-based spelling check is an essential approach for detecting spelling mistakes. This technique compares input words with a predefined vocabulary dictionary, words that are missing from dictionary are considered as potential spelling mistake and moves to next phases correction steps such as candidate ranking and generation. According to HassanPour Zonoozi & Seydi (2022) most of the studies highlighted dictionary-based methods are most effective because of their simplicity, ability to provide accurate mistake detection and interpretability when a suitable vocabulary resource is available.

Conventional spelling checkers only rely on general purpose dictionaries. Specific domain-based spelling check system can improve accuracy of correction by using specialized vocabularies extracted from target corpus (Idrissi-Yaghir et al., 2023). This method is important for scientific documents where it contain specialized abbreviations, technical vocabulary and terminology which may not exist in standard dictionaries. That's why, this study builds a custom dictionary containing words from arXiv corpus which allows the spelling checker to detect scientific terms while maintaining effective detection of spelling mistakes. The created vocabulary provides an efficient basis for generating candidate using minimum edit distance and following correction based on context using probabilistic language modeling.

1.1.3. Minimum Edit Distance

After detecting incorrect spelled words, this system determines which dictionary words are most likely to be correct. According to GUEDDAH (2021), many recent studies proved that combined edit distance with contextual language models can improve spelling correction accuracy significantly.

This study uses the Minimum Edit Distance (Damerau-Levenshtein Distance) which is one of the most widely used methods for spelling checker system which helps to calculate the minimum number of insertion, deletion, transportation and substitution tasks that requires to change one word into another (Damerau, 1964). For example, an incorrect word (mahine) needs only one insertion operation to convert to correct word (machine) which results in a small edit distance. These lower edit distance candidate words are thought as to be the intended spelling.

This minimum edit distance method is simple, efficient and effective, that's why it remains one of the most commonly used methods in modern spelling correction system.

1.1.4. Unigram and Bigram Language Models

For correcting non-word mistakes, edit-distance works effectively but it cannot distinguish between words which are used in the wrong context. That's why probabilistic language models are used for evaluating contextual information. Even though these methods originated decades ago, they still play an essential part in many modern spelling correction systems. And will continue to be the basis of modern methods and language models (Wang et al., 2021)

According to Bryant et al (2022) probabilistic models like unigram and bigram are used to improve contextual accuracy by allowing the system to choose the most accurate correction according to its surrounding context. Unigram language model calculates the probability of each word depending on their frequency inside a corpus which allows the most frequent words to have higher probabilities. Then bigram language model upgrades this concept by considering pair-words which also calculates the probability of one word that follows another in natural language.

As bigram model assigns higher probability to the correct sentence, it enables the system to suggest appropriate corrections contextually. In this study, the system can improve the accuracy of detecting both non-word and real-word mistakes by combining edit-distance calculations with probabilistic language models.

1.1.5. GUI of Spell Checker

For enhancing the usability of the spelling checker system, this study used graphical user interface (GUI) which provides an intuitive environment for user interaction. GUI enables users to input text, perform spelling checks, browse in generated dictionary and suggest a suggestion list of corrections through a visual interface. With all these use cases it improves the accessibility of the system and makes it suitable for users with different levels of technical expertise.

This project used python's Thinter library for developing the GUI as it is cross-platform, light-weighted and included in the standard python distribution. Its interface contains a text input panel, dictionary exploration panel and a suggestion list. All these options enable users to interact with the system efficiently. After entering text in the text input panel, it highlights the spelling mistakes within the editor and by clicking on highlighted word it will display a list of suggested corrections generated by the language model and edit-distance algorithm. This architecture makes the user experience better by making the correction process interactive, easy and transparent to apply.

1.2 Summary

To conclude this chapter, discuss the spelling checker system and the candidate techniques used for implementing the proposed framework. Staring with tokenization that prepares the input text for processing, then dictionary-specified spell checker detects potential spelling mistakes and lastly minimum edit distance method creates candidate corrections based on word similarity. Unigram and bigram language models were used to improve contextual accuracy by evaluating the probability of candidate words within the surrounding texts. And lastly, GUI provides a user-friendly and easily accessible interface for users to interact with the system. Altogether, this approach made this framework solid for developing a spelling checker system which can manage scientific text while fulfilling the functional requirements of this assignment.

2. Edit Distance

Edit distance is used for measuring the similarity between two words, that's why it is one of the most common techniques in spelling checker systems. It can estimate the minimum number of editing operations generally like insertion, deletion, transportation and substitution required to transform one word to another. The assumption of this algorithm is that words with smaller distances are more likely to be present with the intended spelling of an incorrect word. Because of efficiency, easy interpretability and simplicity, edit distance method have become one of the most important methods in systems like spell checking, machine translation and information retrieval (Levenshtein, 1966).

In this spelling checker system edit distance is used basically in candidate generation step, generating multiple candidate words then it ranks them according to their similarity to the incorrect word. Wang et al. (2021) highlighted that edit distance method work as a primary foundation for traditional spelling checker system before combining it with probabilistic methods to find out the most appropriate correction.

2.1 Types of Edit Distance Method

There are many types of edit distance techniques which are introduced to improve spelling checker systems accuracy. These techniques share the same objective to measure similarity between two words but still they differ in the way transformation cost is calculated and the types of editing operations considered. According to Wang et al. (2021) the most widely used edit distance techniques for spelling checker systems are:

a. Levenshtein Distance
b. Damerau-Levenshtein Distance
c. Weighted Edit Distance

For selecting the suitable edit distance technique, first need to understand the nature of the system, the expected types of spelling mistakes and also need to understand the computational complexity of the algorithm.

a. Levenshtein Distance: The most widely used variation of edit distance is Levenshtein Distance which serve as the basis for many spelling checker systems. According to Levenshtein (1966) this method determines the minimum number of required insertions, substitutions and deletions to change one word to another. Each editing operation is equal to cost of one which is why this algorithm calculates the similarity between two words simply and computationally efficiently.

For example, an incorrect word (lerning) needs only one insertion operation to convert to correct word (learning) which results in a small edit distance. These lower edit distance candidate words are thought as to be the intended spelling before applying probabilistic

ranking method. The formula for Levenshtein Distance is given in equation (1) (Levenshtein, 1966):

$$D(i,j) = \begin{cases} max\ (i,j) \\ min \begin{cases} D(i-1,j)\ +\ 1 \\ D(i,j-1)\ +1 \\ D(i-1,j-1)\ +cost \end{cases} & if\ i = 0\ or\ j \\ = 0 \end{cases}$$

…… (1)
(Levenshtein, 1966)

Where,

- Insertion = D (i, j-1) +1
- Deletion = D (i-1, j) + 1
- Substitution = D (i-1, j-1) + cost

This formula calculates the minimum editing cost by trying all possible transformations between two words.

b. Damerau-Levenshtein Distance: The upgraded version of Levenshtein algorithm is Damerau-Levenshtein Distance by including an extra editing operation known as transportation. This operation swaps two adjacent characters (Damerau, 1964). It was proposed after seeing that typing mistakes that accidentally reverse neighboring letters.
For example, an incorrect word (Sceince) needs only one transportation operation of i and e to convert to correct word (Science) in one single operation. If it was standard Levenshtein technique, then it would have treated this as two substitutions. So, it proves that Damerau- Levenshtein creates smaller edit distance for common typing mistakes and provides more accurate correction candidates. The formula is given in equation 2 (Damerau, 1964):

$$D(i,j) = \begin{cases} max\ (i,j) \\ min \begin{cases} D(i-1,j)\ +\ 1 \\ D(i,j-1)\ +1 \\ D(i-1,j-1)\ +\ cost \\ D(i-2),j-2)\ +1 \end{cases} & if\ i = 0\ or\ j \\ = 0 \end{cases}$$

…… (1)
(Damerau, 1964)

Where,

- Insertion = D (i, j-1) +1
- Deletion = D (i-1, j) + 1
- Substitution = D (i-1, j-1) + cost
- Transportation = D (i-2, j-2) + 1

The formula for this algorithm is extended version of Levenshtein by including an additional transportation logic.

c. **Weighted Edit Distance:** Another upgraded version of edit distance algorithm is Weighted Edit Distance which contains different editing operations that are assigned with different cost rather than treating every operation equally. The weight actually estimates based on the probability of specific mistakes.
For example, changing the character m with n will cost lower penalty than changing m with x because adjacent keyboard characters are more likely to be typing error. The formula for weighted distance is given below in equation 3 (Brill & Moore, 2000):

$$D(s,t) = \sum_{i=1}^{n} w_i \; x \; op_i \qquad \text{.....(3)}$$

(Brill & Moore, 2000)

Where,

- D (s,t) = wight of edit distance between string s and t.
- w_i = cost assigned to the ith editing operation.
- op_i = editing operation
- N = total number of editing operations required to transform one to aonther.

Weighted edit distance gives more realistic modeling for human typing behavior and improves candidate ranking but determining appropriate weights requires additional training data or cost metrices which make the algorithm more complex to implement (Brill & Moore, 2000).

2.2 Comparison between Edit Distance Types

Table 1 contains the details of the three editing distance types differ based on their computational complexity, supported editing operations and suitability for spelling checker system.

Techniques	Operations	Advantages	Limitations	Suitable or not
Levenshtein Distance	Insertion, substitution and deletion	Efficient and easy to implement.	Cannot manage transportation mistakes directly.	Not fully suitable.
Damerau-Levenshtein Distance	Insertion, substitution, deletion and transportation	Better for correcting typing errors.	Computationally complex.	Yes
Weighted Edit Distance	Weighted insertion, substitution, deletion and transportation	Creates more reliable candidate ranking.	Need pre-defined weights or training data	Future enhancement.

Table 1: Comparison Table of Three Type of Edit Distance.

Because of good balance between computational efficiency, simple implementation and correction accuracy, Damerau-Levenshtein Distance was suitable for this project. But Levenshtein Distance and Weighted Edit Distance were the better algorithms which can get higher accuracy for specific mistake types, but it comes along with extra computational complexity and needs extra resources which are more than the scope of the proposed system. As Unigram and Bigram language models are being combined with the Damerau-Levenshtein Distance, it will help the system to improve correction accuracy by contextual probability. That's why Damerau-Levenshtein Distance was selected for this study.

3. Formulation and System Design

Here this section will discuss the proposed spelling checker system developed for scientific text. This system is implemented by using traditional NLP techniques with probabilistic language modeling for detecting and correcting non-word and real-word spelling mistakes. It's not like conventional dictionary-based spell checker system that only check whether a word belongs in vocabulary. For improving accuracy of correction, this system is using contextual information. The overall design of the proposed spelling checker system is shown in **Figure 1**.

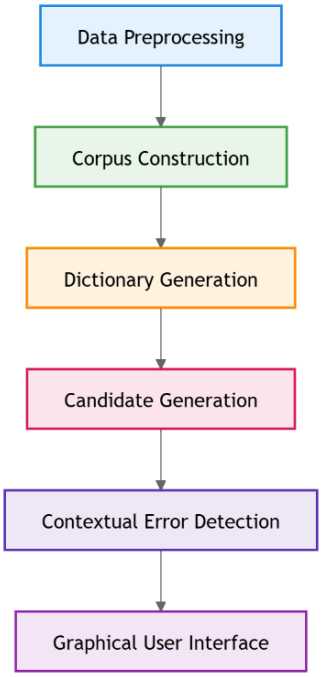


Figure 1: Six major stages of Spelling Checker System.

The system consists of six major stages: 1) data preprocessing (cleaning and tokenization), 2) corpus construction, 3) dictionary generation, 4) candidate generation, 5) contextual error detection and 6) graphical user interface (GUI). According to Eisenstein (2019), this formulation simplifies implementation, improves maintainability and allows extra NLP methods to be combined in future improvements.

3.1 Dataset Selection and Corpus Construction

This study is using the arXiv Metadata dataset from Kaggle which contains millions of scientific research publications from multiple disciplines. This was selected because of domain-specific technical expression and structured academic language. The titles and abstracts of these publications provide large collection of scientific vocabulary which represent academic writing significantly than a general English dictionary. This enables the system to:

- Recognize scientific words.
- Avoid false positives on technical words.
- Provide meaningful contextual correction suggestions.

The system's ability to understand and handle scientific spelling mistakes effectively by using this dataset.

3.2 Overall System Architecture

System follows a modular pipeline design:

a. **Tokenization:** It starts with constructing a domain-specific corpus like arXiv scientific dataset which contains title and abstract of each research paper. The input text gets processed by using custom tokenizer that extracts alphabetic words while converting it to lowercase and removes special characters, punctuations, numbers and unnecessary symbols.
b. **Corpus-Driven Language Model:** These generated individual words from tokenization are used to construct the system dictionary with both unigram and bigram language models. These probability models form the basis for detecting context-based mistakes and ranking candidates. Also, this scientific corpus helps the system to recognize technical terminology and create appropriate corrections for academic writing.
c. **Spelling Checker Engines:** The engine contains Minimum Edit distance to generate candidate corrections for non-word mistakes, Bigram language model to check contextual fit and detect real-word mistakes, Confusion sets and Grammar rules to improve real-word mistake detection and lastly candidate ranking helps to combine edit distance and probabilistic scoring.
d. **Graphical User Interface:** The system developed its graphical user interface by using python's tkinter library which provides interactive environment for users. The way it is designed provides the users with finding mistakes easily, explore correction options and simplifies the interaction with system without requiring any command-line operations.

3.3 Detection and Correction Strategy

The system of this study combines multiple NLP methods for improving the spelling correction accuracy. It does not rely on dictionary lookup only, also integrates contextual probability and textual similarity to identify different types of spelling mistakes.

a. **Non-word mistakes detection**: Extracted tokens from the user input is compared with the words from corpus dictionary for detecting non-word mistakes. The words which were not found in the dictionary were identified as spelling mistakes immediately. Also, Damerau-

Levenshtein Distance was applied to create candidate words that require the minimum number of editing operations for correction.

b. **Real-word mistakes detection:** It's not possible to detect real-word mistakes by using dictionary lookup. Because the word that is used incorrectly is still valid word in the vocabulary entry. That's why the proposed system used bigram language model to evaluate contextual relationship between consecutive words. When bigram probability of a word is very low with its preceding word, then the system considers the surrounding context to be unusual and creates appropriate candidate words for suggestion. These words are ranked according to both edit distance and contextual probability for determining the most appropriate correction.

This system can correct both spelling mistakes and context-based word mistakes by combining textual similarity with probabilistic language modeling.

3.4 Graphical User Interface Design

The interface for users has been designed according to the requirements while maintaining simplification of use. The GUI of this system uses four primary components:

- A text editor which allows user to input text (maximum limit 500 characters)
- A dictionary panel which shows all corpus words in sorted way and provides a search bar for looking at specific words.
- A suggestion list for displaying correction words with their minimum edit distance value.
- Spell checking Button which started the spelling checker process and highlights incorrect words.

Figure 2 contains the screenshot of this system GUI design.

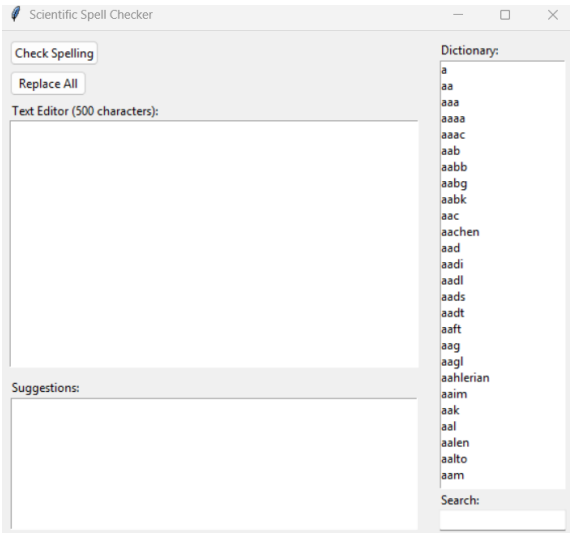


Figure 2: Layout of GUI of the system.

Incorrect words will be highlighted in the input text to improve readability. When user selects highlighted mistakes one by one, it will show the correction suggestions which will allow user to replace the incorrect word effectively.

3.5 Functional Flowchart of the System

Figure 3 represents the working procedure of the spelling checker system.

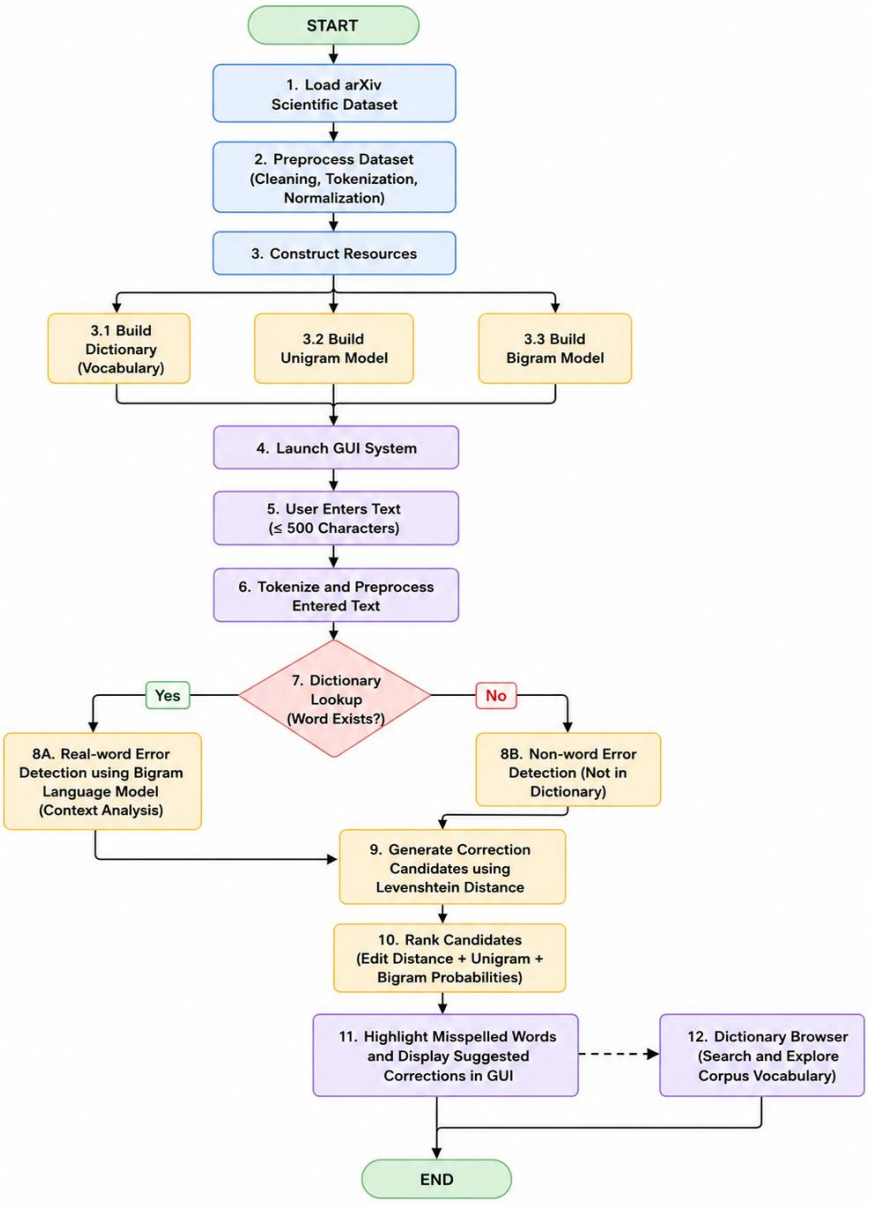


Figure 3: Workflow of the Spelling Checker System.

3.6 Summary

To conclude this study combines preprocessing, tokenization, dictionary lookup, minimum edit distance, unigram and bigram language models together to develop a better spelling checker system for scientific text. The use of specific domain corpus helped to improve the recognition of technical words and by using contextual probability it helped to enable the detection of both non-word and real-word spelling mistakes. And finally, GUI provided a simple interface that allowed users to input text, detect spelling mistakes, apply appropriate corrections and explore the corpus dictionary efficiently.

4. Implementation

4.1 Development Environment and Libraries

This system was developed in Python by using combined standard libraries. From **Figure 4**, it describes that **re** library was used for regular expression-based tokenization and **json** was used for processing the large jason format arXiv metadata dataset. Classes of **collections** module, **counter** and **defaultdict** were used for maintaining unigram and bigram frequency counts efficiently. During candidate scoring, **math** library was used for logarithmic probability calculations. Finally, for the GUI of desktop-based spelling checker system, we used the built in **Tkinter** library with **ttk** and **messagebox**.

```
# Import required liberies
import json, re, math
from collections import Counter, defaultdict
import tkinter as tk
from tkinter import ttk, messagebox
```

Figure 4: Required libraries for spelling checker system.

4.2 Text Processing and Tokenization

Figure 5 represents the tokenization function tokenize_with_positions(). It was used for preparing the input text for NLP processing. A regular expression pattern (**re**) was used for extracting alphabetic word sequences by removing punctuation, numbers and special symbols.

Then extracted words were converted to lowercase so that it can ensure consistency between the user input and the vocabulary learned from the training corpus. Also, original form of each word was retained separately so that NLP model can operate on normalized tokens while keeping the original text representation for the GUI. It also keeps track of the start and end character positions of every token because it will help to find and highlight detected spelling mistakes directly in the text editor. This step makes sure that only linguistically relevant tokens are passed to subsequent stages.

```
# Regex pattern to extract alphabetic words
TOKEN_PATTERN = re.compile(r"[A-Za-z]+")
```



```python
# TOKENIZATION
# Extract alphabetic sequences only
# Convert everything to lowercase

def tokenize_with_positions(text):
    tokens = []          # lowercase tokens for NLP
    originals = []       # original words for GUI
    positions = []       # (start, end) positions for highlighting

    for match in TOKEN_PATTERN.finditer(text):
        start, end = match.span()
        word = match.group()

        tokens.append(word.lower())      # used lowercase for NLP model
        originals.append(word)           # GUI keeps original casing
        positions.append((start, end))   # highlight exact characters

    return tokens, originals, positions
```

Figure 5: Tokenization function of Spelling Checker System.

4.3 Corpus Construction and Dictionary Generation

This study is using a scientifics-specific corpus created from arXiv publications. The system reads the json dataset line by line which helps to provide more memory-efficient for processing a large dataset. The corpus is tokenized, cleaned and transformed into a vocabulary list containing scientific terms which is stored in a Python **set** function (**Figure 6**) that helps to enable lookup operation. Also, the corpus was used to create unigram and bigram frequency distribution so that it can serve as the basis for probabilistic language modeling.



```python
        self.vocab = set()
```

Figure 6: Dictionary Generation function of Spelling Checker System.

4.4 Language Model Construction

Figure 7 describes that this system develops a statistical language model using **LanguageModel** class which computes unigram and bigram frequency data. Here **defaultdict** and **counter** from **collections** were used to storing the frequency counts. The vocabulary was grouped into buckets so that it can improve the efficiency of candidate generation. Then laplace smoothing was used to bigram probabilities for ignoring zero-probability problems which helps the language model to continue scoring candidate correction even when specific pair of words were not observed in the training corpus.

```
class LanguageModel:
    def __init__(self):
        self.unigram = Counter()
        self.bigram = defaultdict(Counter)
        self.vocab = set()
        self.total = 0
        self.V = 0
        self.length_buckets = defaultdict(list)
```

Figure 7: Language Model function of Spelling Checker System.

4.5 Minimum Distance and Candidate Generation

Figure 8 shows that this study used **Damerau-Levenshtein Distance** as the main NLP method for creating correction words for non-word spelling mistakes. This method determines the minimum number of required insertions, substitutions and deletions to change one word to another. The algorithm was used for optimization carefully so that it can ignore unnecessary recompilation and make sure that it works smoothly even when processing large vocabulary.

Then candidate generation were optimized by minimizing the search to dictionary whose lengths fall with a small range around the incorrect word. Candidates with one or two edit distances are being retained and subsequently ranked by using combined score of unigram probability, bigram context probability and edit distance penalties. This method makes the system prioritize corrections that are both contextually appropriate and orthographically similar. Also, this whole approach through a dedicated function makes the candidate generation process reusable and clear.

```
# MINIMUM EDIT DISTANCE (Damerau-Levenshtein

def damerau_levenshtein(s1, s2):
```

Figure 8: Damerau-Levenshtein function of Spelling Checker System.

4.6 Detection of Non-Word and Real-Word Mistakes

Figure 9 describes that this system checks two types of spelling mistakes: non-word and real-word spelling mistakes. First, non-words are detected when input words are not present in the dictionary learned from arXiv corpus. And to solve this problem, the system uses Damerau-Levenshtein distance to generate possible corrections and ranks them using statistical language model.

Then the system detects real-word mistakes where the word is spelled correctly but it is used in the wrong context. A predefined set of commonly confused words are combined with additional grammar-based rules and bigram probability technique so that it compares the probability of the detected word with its alternative within the surrounding context. All these approaches combined provide a better comprehensive approach than conventional dictionary spell checking only.

```
# SPELL CHECKER
# Non-word errors (misspellings)
# Real-word errors (valid words used incorrectly)

class SpellChecker:
    def __init__(self, lm):
        self.lm = lm
```

Figure 9: Spellchecker function of Spelling Checker System.

4.7 Graphical User Interface (GUI)

Figure 10 represents that an interactive graphical user interface was implemented to make the system accessible through a text editor. It contains a text input area, spelling checker button, suggestion list and a searchable dictionary panel.

```
# TKINTER GUI

class SpellCheckerGUI:
    def __init__(self, root, checker, lm):
        self.root = root
        self.checker = checker
        self.lm = lm

        self.errors = []
        self.token_positions = []
        self.original_tokens = []
        self.selected_error = None

        self.root.title("Scientific Spell Checker")
        self.build_gui()
```

Figure 10: GUI function of Spelling Checker System.

When a user input text (maximum limit 500 characters) in the input box, it gets tokenized and processed by the spell-checking function and detects mistakes, highlight them directly in the text editor using dedicated visual tag. Then user can select incorrect words one by one to view the ranked correction suggestions.

The selected suggestion correct word can be applied directly to the text. After applying correction is applied spelling checker process executes again so that updated text can be re-evaluated. A dictionary panel which shows all corpus words in sorted way and provides a search bar for looking at specific words.

4.8 Efficiency and Optimization of Implementation

Several approaches were tried to improve the efficiency of the system. But the best approaches for this system were:

- The training corpus limited to 2,00,000 records for providing balance between corpus size and processing requirements.

- Reduced the need of loading the complete dataset into memory by processing it sequentially.
- The use of Python sets function for dictionary lookup.
- The use of Counter for efficient frequency counts.
- Ignoring repeated tokenization.
- The use of length-based buckets for narrowing the search space significantly during edit-distance-based candidate generation.
- Candidate search is constrained by considering words in limited length range (like edit distance one or two)
- The use of modular class-based architecture for reusability and clarity.

All these optimizations provide the system to perform efficiently when processing a large and complex terminology dataset and multiple correction candidates.

4.9 Summary

This implementation architecture makes the spelling checker system structured, well-documented and optimized for performance. Here the system is developed by combining python coding with advanced NLP methods which includes tokenization, edit distance, probabilistic language modeling and context-based ranking. The interactive GUI has enhanced usability and provides a simple environment for detecting and correcting both non-word and real-word spelling mistakes.

5. Results

In this section, we evaluated the developed scientific spelling checker system by using different types of test cases so that we can assess its capability to detect non-word spelling mistakes, recognize scientific domain words, handle multiple mistakes, identify real-word mistakes and provide correction list through GUI. It does not only check whether incorrect words are being detected but also check whether it is giving appropriate suggestions and avoiding incorrectly flagging valid words.

5.1 Test Case 1: Detection of Non-Word Mistake

Figure 11 shows the input and output result of test case 1.

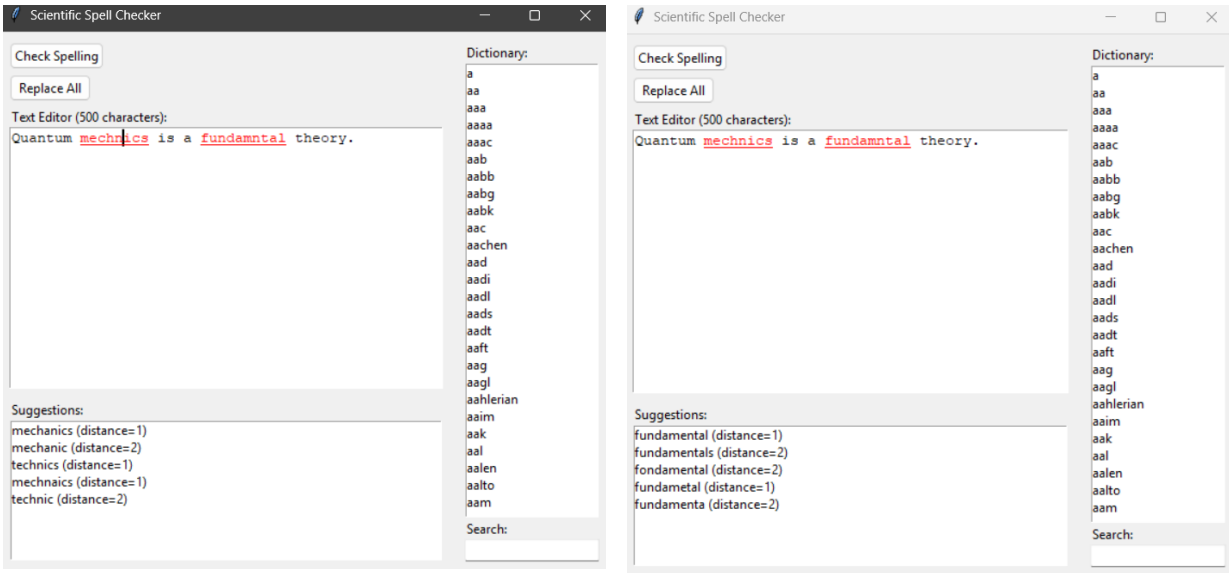


Figure 11: Spelling Mistakes of Input Text of Test Case 1.

So, the first input was “Quantum mechnics is a fundamntal theory”. Here, the system highlighted ‘mechnics’ and ‘fundamntal’ in red as it is a spelling mistake. The words are not present in the vocabulary learned from scientific corpus. Then candidate generation method used Damerau-Levenshtein distance to find words with a small edit-distance operation and provided correction words in the suggestion box.

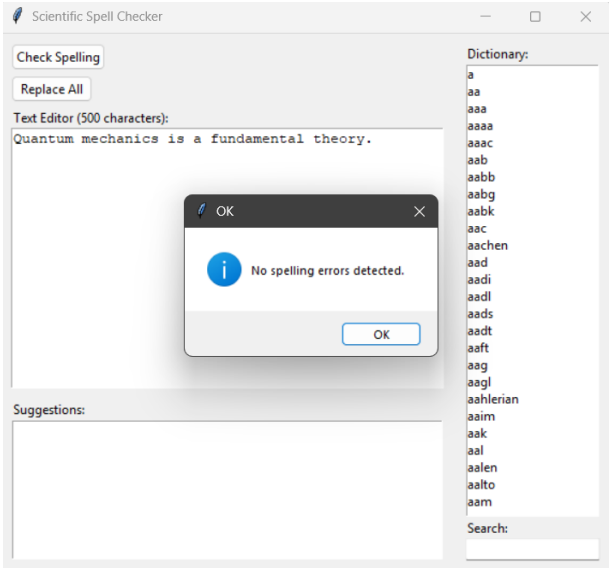


Figure 12: Correction Output of Test Case 1.

By clicking replace-all button, **Figure 12** shows the correct sentence in the text editor. This proves that the system can detect conventional spelling mistakes and can provide appropriate correction suggestions while leaving the remaining correct words unflagged.

5.2 Test Case 2: Detection of Real-Word Mistake Using Context

Figure 13 shows the input and output result of test case 2.

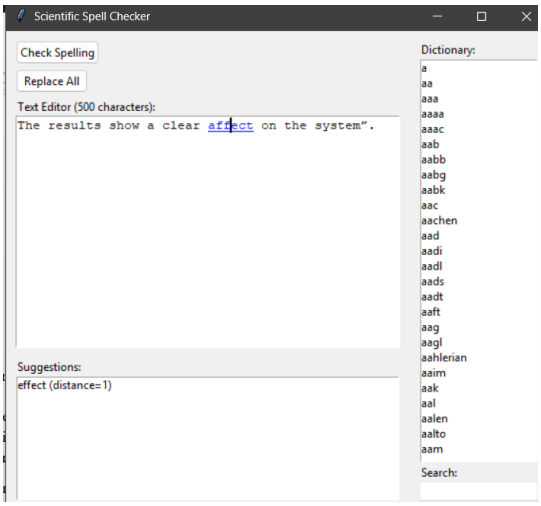


Figure 13: Spelling Mistakes of Input Text of Test Case 2.

So, the first input was "The results show a clear affect on the system". Here, the system highlighted 'affect' in blue because it is a word which is present and valid in vocabulary. This mistake is highlighted by using contextual-error detection method which compares statistical probability of

the highlighted word with its predefined alternative ‘effect’ with surrounding word context and bigram probabilities.

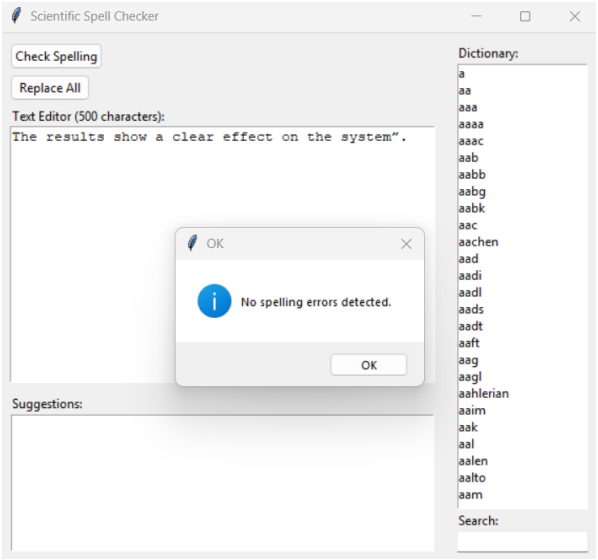


Figure 14: Correction Output of Test Case 2.

By clicking replace-all button, **Figure 14** shows the correct sentence in the text editor. This proves that the system is not limited to finding words that are absent from the vocabulary but also can detect real-word mistakes which are inappropriate within its context and can provide appropriate correction suggestions while leaving the remaining correct words unflagged.

5.3 Test Case 3: Managing Scientific Domain Vocabulary

Figure 15 shows the input and output result of test case 3.

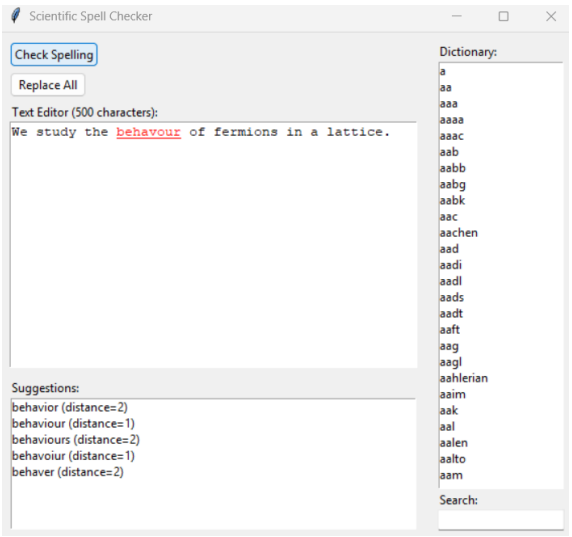


Figure 15: Spelling Mistakes of Input Text of Test Case 3.

So, the first input was “We study the behavour of fermions in a lattice”. Here, the system highlighted ‘behavour’ in red as it is a spelling mistake. The word ais not present in the vocabulary learned from scientific corpus. Then candidate generation method used Damerau-Levenshtein distance to find words with a small edit-distance operation and provided correction words in the suggestion box. Also importantly, fermions’ is considered as valid word in the dictionary and as result it was not highlighted incorrectly.

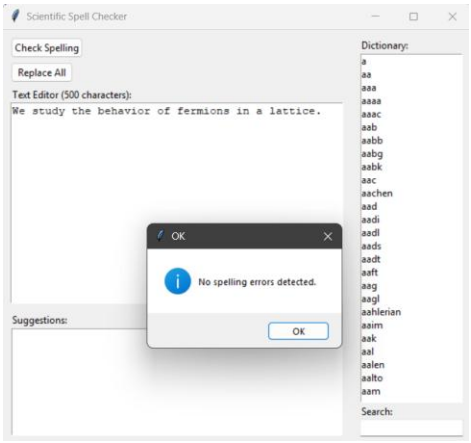


Figure 16: Correction Output of Test Case 3.

By clicking replace-all button, **Figure 16** shows the correct sentence in the text editor. This proves that the system works well with the scientific writing context because scientific documents frequently contain special terminology. By training the model with scientific corpus it allowed valid specific-domain terms to be retained rather than treating it incorrectly as spelling mistake.

5.4 Test Case 4: Detecting Mixed Mistake Types

Figure 17 shows the input and output result of test case 4.

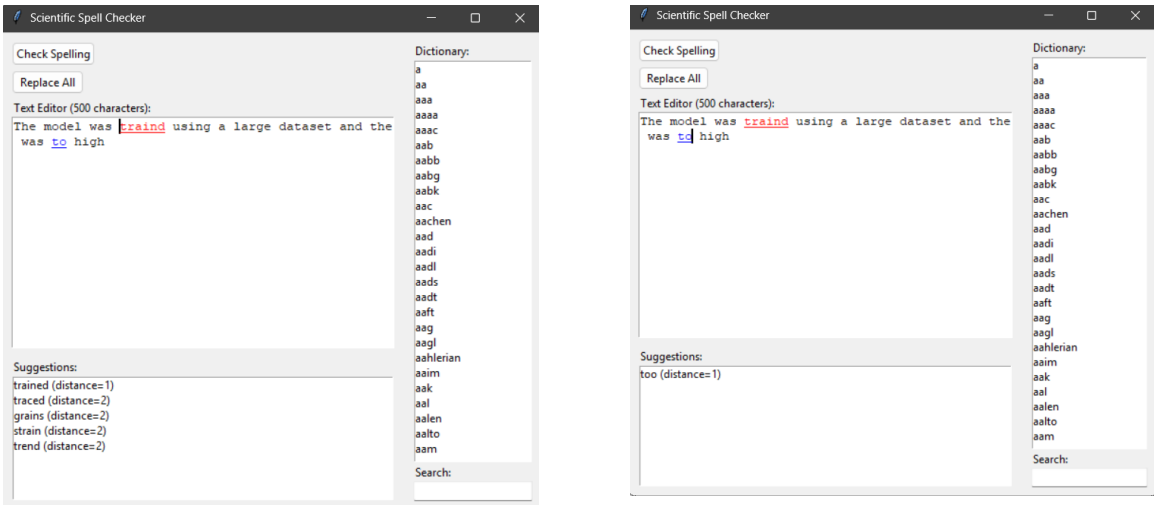


Figure 17: Spelling Mistakes of Input Text of Test Case 4.

So, the first input was “The model was traind using a large dataset and the was to high”. Here, the system highlighted ‘traind’ in red as it is a spelling mistake. The word ais not present in the vocabulary learned from scientific corpus. Then candidate generation method used Damerau-Levenshtein distance to find words with a small edit-distance operation and provided correction words in the suggestion box. Also, the second error ‘to’ is considered as grammar-rule and contextual mistake and suggested ‘too’ as the correction.

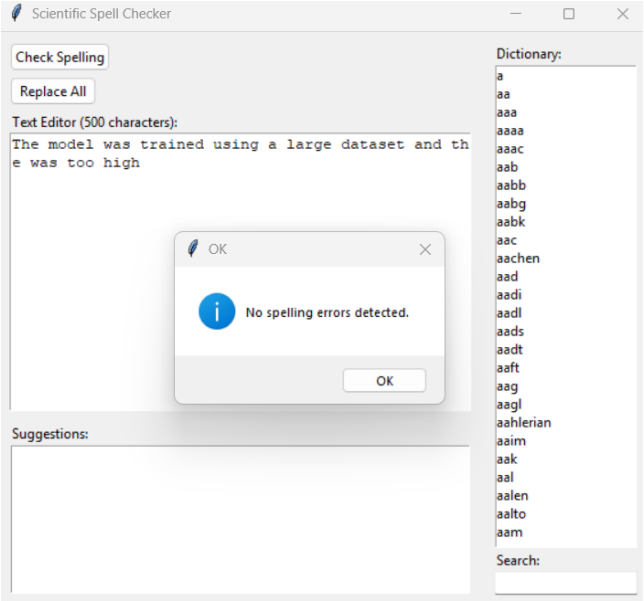


Figure 18: Correction Output of Test Case 4.

By clicking replace-all button, **Figure 18** shows the correct sentence in the text editor. This proves that the system works well by combining multiple NLP-based spelling mistake detection techniques in one single input rather than depending on one technique.

Above all these test cases represent that the system meets all the functional requirements and provides better accuracy, robustness and usability. This combined advanced framework of NLP methods and well-prepared GUI make sures that the system works effectively in real scientific writing scenarios. The test cases also prove that the system works better than simple-dictionary-based spelling checker by considering both the spelling form of a word as well as its surrounding linguistic context as well as.

6. Conclusion

To conclude, this study developed a scientific spelling checker system combining multiple NLP methods including tokenization, Damerau-Levenshtein edit distance, unigram and bigram language modeling, contextual analysis and grammar-based rules. The system can detect both non-word spelling mistakes and real-word spelling mistakes while providing contextually relevant correction suggestions. The integration of the GUI provided a user-friendly platform for detecting, reviewing, exploring and correcting spellings in scientific text. Overall, this study explains how NLP methods can be combined effectively for developing a better spelling checker system specifically for scientific and technical writing.

7. References

Brill, E., & Moore, R. C. (2000). An improved error model for Noisy channel spelling correction. *Proceedings of the 38Th Annual Meeting of the Association for Computational Linguistics (ACL)*. https://aclanthology.org/P00-1037.pdf

Bryant, C., Yuan, Z., Qorib, M. R., Cao, H., Ng, H. T., & Briscoe, T. (2022). Grammatical error correction: A survey of the state of the art. *arXiv:2211.05166 [Cs]*, *1*. https://arxiv.org/abs/2211.05166

Cassee, N., Andrei Agaronian, Constantinou, E., Novielli, N., & Serebrenik, A. (2024). Transformers and meta-tokenization in sentiment analysis for software engineering. *Empirical Software Engineering*, *29*(4). https://doi.org/10.1007/s10664-024-10468-2

Dagan, G., Synnaeve, G., & Baptiste Roziere. (2024). Getting the most out of your tokenizer for pre-training and domain adaptation. *PMLR*, *235*, 9784–9805. https://proceedings.mlr.press/v235/dagan24a.html?

Damerau, F. J. (1964). A technique for computer detection and correction of spelling errors. *Communications of the ACM*, *7*(3), 171–176. https://doi.org/10.1145/363958.363994

Eisenstein, J. (2019). Introduction to natural language processing. In *MIT Press*. https://mitpress.mit.edu/9780262042840/introduction-to-natural-language-processing/

GUEDDAH, H. (2021). Efficient weighted edit distance and n-gram language models to improve spelling correction of Segmentation errors. *International Journal of Advanced Computer Science and Applications*, *12*(12). https://doi.org/10.14569/ijacsa.2021.01212113

HassanPour Zonoozi, M., & Seydi, V. (2022). A survey on adversarial domain adaptation [Interview]. *Neural Processing Letters*, *55*. https://doi.org/10.1007/s11063-022-10977-5

Idrissi-Yaghir, A., Henning Schäfer, Bauer, N., & Friedrich, C. M. (2023). Domain adaptation of Transformer-Based models using unlabeled data for Relevance and polarity classification of German customer feedback. *SN Computer Science*, *4*(2). https://doi.org/10.1007/s42979-022-01563-6

Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions and reversals. *Soviet Physics Doklady*, *10*(8), 707–710. https://cir.nii.ac.jp/crid/1572824500689070336?

Wang, Y., Wang, Y., Dang, K., Liu, J., & Liu, Z. (2021). A comprehensive survey of Grammatical error correction. *ACM Transactions on Intelligent Systems and Technology*, *12*(5), 1–51. https://doi.org/10.1145/3474840

Task 2 – Text Classification System (Sentiment Analysis)

1. Introduction

Text classification is one of the core applications of Natural Language Processing (NLP), covering tasks such as spam detection, topic labelling, fake-review detection and sentiment analysis. This study focuses on sentiment analysis: automatically predicting whether a piece of free text expresses a positive or negative opinion. Sentiment classification is widely used in real applications such as monitoring product reviews, analysing social media opinion and summarising customer feedback, which makes it a practically important and well-studied NLP task.

Movie reviews present a particularly demanding version of this task compared to short-text sentiment problems (e.g. tweets), because reviews are long-form, frequently mix positive and negative statements about different aspects of the same film, and often rely on sarcasm, negation and comparison to express an overall opinion. This study therefore uses the IMDB Dataset of 50K Movie Reviews (Maas et al., 2011), a long-established benchmark dataset for sentiment classification, to design, build, tune and evaluate a supervised text classification system.

1.1 Introduction to Candidate Techniques

There are several interconnected techniques that work together in a supervised text classification pipeline: text preprocessing and feature extraction to convert raw text into a form a model can learn from, a choice of classification algorithm(s) to learn the mapping from features to sentiment label, and an evaluation methodology to measure and compare how well each algorithm performs. This study's system was developed using TF-IDF feature extraction, three families of classification algorithm spanning classical machine learning and deep learning (Naive Bayes and Logistic Regression, Support Vector Machines and Random Forests, and a Bidirectional LSTM), and standard classification evaluation metrics (accuracy, precision, recall and F1-score).

1.1.1 Text Preprocessing and Feature Extraction

Before any model can be trained, raw review text must be cleaned and converted into numeric form. This study removes residual HTML tags, lowercases all text, strips punctuation and digits, and removes English stopwords using NLTK's stopword list, since these very common words (e.g. "the", "is", "and") carry little sentiment information but dominate raw word-frequency counts. The cleaned text is then converted into numeric features using Term Frequency–Inverse Document Frequency (TF-IDF) weighting, which scores each word (and, in this study, each word pair) by how important it is to a specific review relative to how common it is across the whole dataset — giving high weight to distinctive, sentiment-carrying terms and low weight to generic terms that appear in almost every review (Pedregosa et al., 2011, describe the TfidfVectorizer implementation used here as part of the widely-used scikit-learn library).

1.1.2 Classical Machine Learning Classifiers

Four classical algorithms were reviewed and implemented. Multinomial Naive Bayes is a simple probabilistic classifier that assumes each TF-IDF feature is conditionally independent of the others given the class label; it is fast and commonly used as a baseline for text classification, although its independence assumption is a known weakness for correlated features such as overlapping unigrams and bigrams. Logistic Regression instead directly learns a discriminative linear decision boundary over the same feature space, without assuming feature independence, and is widely reported as a strong baseline for sparse, high-dimensional text data. Support Vector Machines (SVM), introduced by Cortes and Vapnik (1995), find the linear boundary that maximises the margin between the two sentiment classes rather than only minimising misclassifications, and were also the classifier used in the original IMDB benchmark paper by Maas et al. (2011). Random Forest (Breiman, 2001), an ensemble of decision trees each trained on a random subset of features and data, was included as a contrasting non-linear approach to test whether tree-based partitioning of the TF-IDF space could outperform the linear models. All four algorithms were implemented using the scikit-learn library (Pedregosa et al., 2011).

1.1.3 Deep Learning Approaches

Unlike the TF-IDF-based classical models above, which treat a review as an unordered "bag" of weighted terms, sequence models can learn from word order and context. This study implements a Bidirectional Long Short-Term Memory (LSTM) network (Hochreiter & Schmidhuber, 1997), a type of recurrent neural network designed to retain information over long input sequences using gated memory cells, reading each review both forwards and backwards to build a richer representation of its content before predicting sentiment. Modern transformer-based language models such as BERT (Devlin et al., 2019) represent a further step beyond recurrent architectures, using self-attention to model context, and have been shown to substantially outperform both classical and LSTM-based approaches on sentiment benchmarks including this one; however, such models require considerably more training data and compute than is practical within the scope of this assignment, so a Bidirectional LSTM was selected as this study's deep learning model instead.

1.1.4 Model Evaluation Metrics

Because the dataset is perfectly balanced between positive and negative reviews (see Section 2), accuracy is a meaningful headline metric, but it was supplemented with precision, recall and F1-score to give a fuller picture of each model's behaviour, following standard practice in sentiment classification research (Pang & Lee, 2008). F1-score, the harmonic mean of precision and recall, was used as the primary metric for selecting the best-performing model and for hyperparameter tuning, since it balances both error types rather than favouring one class over the other.

1.2 Summary

To conclude this chapter, this study reviewed the candidate techniques used for building a supervised sentiment classification system: TF-IDF feature extraction, four classical machine learning algorithms (Naive Bayes, Logistic Regression, SVM, Random Forest) and one deep learning architecture (Bidirectional LSTM), evaluated using standard classification metrics. The remainder of this report covers the dataset and exploratory analysis (Section 2), the system's design (Section 3), its implementation (Section 4), and its evaluated results (Section 5).

2. Dataset and Exploratory Data Analysis

Selecting an appropriate dataset and understanding its properties before modelling is essential in any supervised learning task, since the choice of dataset directly shapes preprocessing decisions, model design and how results should be interpreted.

2.1 Dataset Selection and Justification

This study uses the IMDB Dataset of 50K Movie Reviews, a cleaned CSV release of the Large Movie Review Dataset originally introduced by Maas et al. (2011). It contains 50,000 movie reviews collected from IMDB, evenly split into 25,000 positive and 25,000 negative examples, with each review labelled only when it expresses a clearly polarised opinion. This dataset was selected over alternatives such as Amazon product reviews or Twitter sentiment corpora for several reasons: its exact class balance avoids the need for imbalance-handling techniques; it is a long-established benchmark, meaning results obtained here can be compared against a large body of previously published work, from the original hybrid bag-of-words/word-vector approach through to modern transformer models; and its long-form, mixed-opinion reviews present a more realistic and difficult classification problem than short-text sentiment tasks.

2.2 Exploratory Data Analysis

Before building any models, the dataset was explored to check its structure, confirm class balance, and understand the vocabulary and phrasing that distinguish positive from negative reviews.

Class Balance

The dataset was confirmed to contain exactly 25,000 positive and 25,000 negative reviews with no missing values in either column, confirming the perfect balance expected from the dataset documentation (**Figure 1)**.

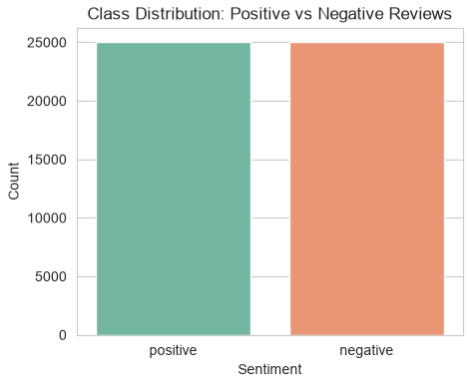


Figure 1: Class distribution of the IMDB 50K dataset.

Review Length Distribution

Word-count statistics show a mean review length of 231.2 words (median 173), ranging from 4 to 2,470 words. **Figure 2** shows the distribution is right-skewed for both classes, with no strong visual difference in length between positive and negative reviews, suggesting review length alone is not a useful predictor of sentiment and that classification must rely on the words used rather than how much is written.

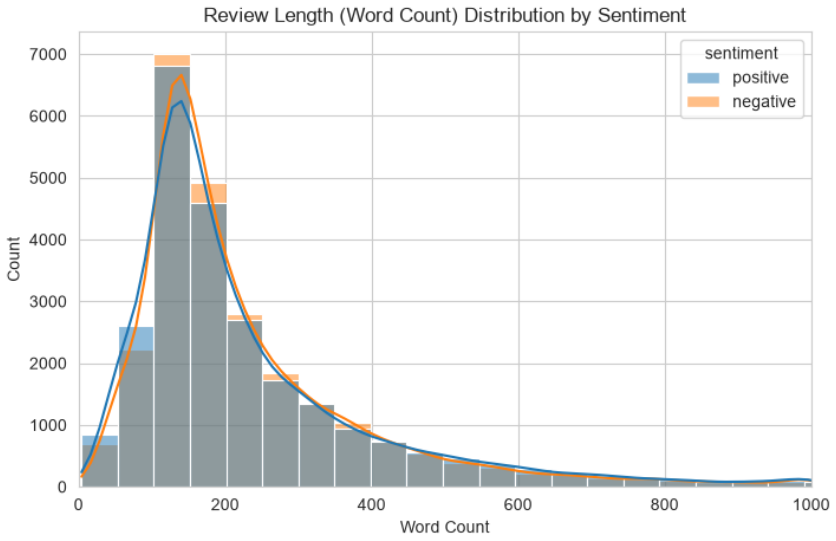


Figure 2: Review length (word count) distribution by sentiment class.

Most Frequent Words per Class

After cleaning (HTML removal, lowercasing, punctuation stripping, stopword removal), the twenty most frequent remaining words were computed separately for each class (**Figure 3**). Both classes are dominated by generic film-review vocabulary ("movie", "film", "character"), which is expected given the domain.

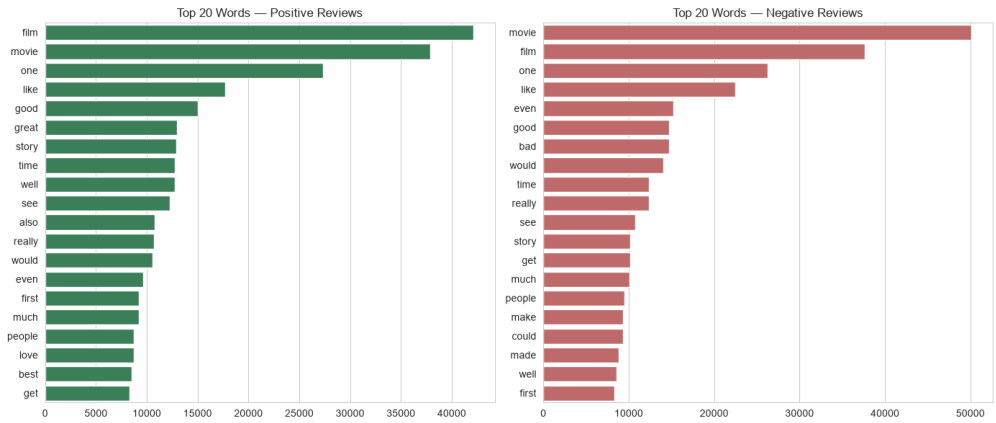


Figure 3: Top 20 most frequent words — positive reviews (left) vs negative reviews (right).

Bigram Analysis

Because single words are dominated by generic vocabulary, two-word phrases (bigrams) were also examined, since they often carry clearer sentiment. Several bigrams were strongly class-indicative — "waste time", "low budget" and "bad movie" appeared only for negative reviews, while "one best", "well done" and "great movie" appeared only for positive reviews. However, phrases such as "ever seen" and "special effects" ranked highly in both classes, since they are used in both superlative-positive constructions ("best movie I've ever seen") and superlative-negative ones ("worst movie I've ever seen") — illustrating a key limitation of bag-of-words and n-gram statistics, which motivated including a sequence model (Section 1.1.3) capable of learning the surrounding context that resolves this ambiguity.

Word Clouds



Figure 4: Word clouds — positive reviews (left, green) and negative reviews (right, red).

2.3 Summary

The IMDB 50K dataset is clean, perfectly balanced, and contains long, linguistically rich reviews that make sentiment classification non-trivial. Exploratory analysis confirmed the dataset's suitability, established baseline statistics used later to sanity-check preprocessing, and identified specific linguistic ambiguities (e.g. shared superlative bigrams across both classes) that directly informed the model design decisions discussed in Section 3.

3. Formulation and System Design

This section discusses the proposed text classification system's design. The system is implemented using traditional and deep-learning-based NLP methods, structured as a modular pipeline covering data preprocessing, feature extraction, model training, hyperparameter tuning and evaluation.

3.1 Preprocessing Pipeline

The following preprocessing pipeline was applied uniformly to all reviews before modelling:

- HTML tag removal: strips residual "
" tags left over from the original scraped HTML.
- Lowercasing: ensures "Good" and "good" are treated as the same token.
- Punctuation and digit removal, plus whitespace collapsing.
- Stopword removal using NLTK's English stopword list, plus removal of single-character tokens.
- Train/test split: an 80/20 stratified split (40,000 train / 10,000 test), preserving the original 50/50 class balance in both partitions, with a fixed random seed for reproducibility.
- TF-IDF vectorisation: fitted on the training split only (vocabulary capped at the 10,000 highest-weighted unigram/bigram terms) and reused to transform the test split, preventing test-set information from leaking into the feature vocabulary.

3.2 Overall System Architecture

The system follows a modular pipeline design:

- Data Loading and Cleaning: the raw CSV is loaded, inspected for missing values, and each review is passed through the cleaning function described in Section 3.1.
- Feature Extraction: the cleaned text is split into training and test sets and converted into TF-IDF vectors (for the classical models) or a tokenised, padded integer sequence (for the LSTM, which needs word order preserved rather than collapsed into a bag of weighted terms).
- Model Training Layer: five models across three groups are trained on the same split for direct comparability — Naive Bayes and Logistic Regression (baseline), Linear SVM and Random Forest (tuned classical), and a Bidirectional LSTM (deep learning).
- Hyperparameter Tuning Layer: GridSearchCV and RandomizedSearchCV are used to tune the classical models; the LSTM is tuned manually with early stopping.
- Evaluation Layer: all five trained models are scored on the same held-out test set using accuracy, precision, recall and F1-score, and compared side by side.

3.3 Model Design and Selection Strategy

Given three group members and the requirement to build n predictive models (n = 3), models were grouped so that each teammate could own a distinct, gradable component while still allowing every model to be evaluated on identical data: (1) a probabilistic baseline (Naive Bayes) alongside a linear discriminative baseline (Logistic Regression); (2) two additional classical models with dedicated hyperparameter tuning (Linear SVM and Random Forest); and (3) a deep sequence model (Bidirectional LSTM) that can learn from word order, addressing the ambiguous-bigram limitation identified during EDA (Section 2.2). This progression allows a like-for-like comparison between classical machine learning and deep learning on the same dataset and split.

3.4 Summary

This system combines TF-IDF feature extraction, multiple classical algorithms, a deep learning sequence model, and a structured tuning and evaluation methodology to build and compare five sentiment classifiers on a shared dataset split. The following section (Section 4) describes how each stage was implemented, illustrated with the actual code and console output produced when running the notebooks.

4. Implementation

4.1 Development Environment and Libraries

The system was implemented in Python using Jupyter notebooks. Data handling used pandas and numpy; text cleaning used Python's built-in re module and NLTK's stopword list; visualisation used matplotlib, seaborn and wordcloud; classical models and TF-IDF feature extraction used scikit-learn (Pedregosa et al., 2011); the deep learning model used TensorFlow/Keras; and joblib and scipy.sparse were used to persist trained models and feature matrices between notebooks.

4.2 Data Loading and Cleaning

The raw CSV (50,000 rows) is loaded with pandas and checked for shape, column types and missing values (none found). Each review then passes through a clean_text() function that strips HTML tags with a regular expression, lowercases the text, removes non-alphabetic characters, collapses whitespace, and removes stopwords and single-character tokens, producing a clean_review column used for all subsequent steps. **Figure 5** shows the dataset loading code and its console output, confirming the expected 50,000 x 2 shape with no missing values.

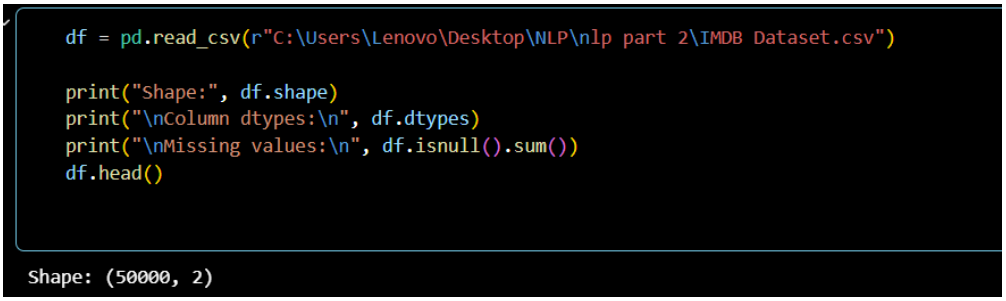


Figure 5: Dataset loading and inspection — code and console output.

4.3 Feature Extraction (TF-IDF Vectorisation)

Sentiment labels are mapped to integers (positive = 1, negative = 0), and the cleaned reviews are split 80/20 into training and test sets using a stratified split to preserve class balance. A TfidfVectorizer (unigrams and bigrams, 10,000-term vocabulary cap) is then fitted on the training text only and used to transform both the training and test text into sparse numeric matrices. **Figure 6** shows this split and vectorisation step, confirming a 40,000/10,000 train/test split and a resulting TF-IDF matrix of shape (40000, 10000).

```python
df['label'] = df['sentiment'].map({'positive': 1, 'negative': 0})

X_train, X_test, y_train, y_test = train_test_split(
    df['clean_review'], df['label'], test_size=0.2, random_state=42, stratify=df['label']
)

print("Train size:", X_train.shape[0], "| Test size:", X_test.shape[0])

tfidf = TfidfVectorizer(max_features=10000, ngram_range=(1, 2))
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)

print("TF-IDF matrix shape (train):", X_train_tfidf.shape)
```

```
Train size: 40000 | Test size: 10000
TF-IDF matrix shape (train): (40000, 10000)
```

Figure 6: Train/test split and TF-IDF vectorisation — code and console output.

These outputs are then saved to disk (**Figure 7**) along with the labels and the fitted vectoriser itself, so the modelling stage does not need to repeat this work.

```python
# Save processed splits for the modeling notebooks (Task 2.2)
import scipy.sparse as sp
import joblib

sp.save_npz('X_train_tfidf.npz', X_train_tfidf)
sp.save_npz('X_test_tfidf.npz', X_test_tfidf)
y_train.to_csv('y_train.csv', index=False)
y_test.to_csv('y_test.csv', index=False)
joblib.dump(tfidf, 'tfidf_vectorizer.pkl')

# Also save the cleaned text splits (needed for the deep learning model's own tokenizer)
X_train.to_csv('X_train_text.csv', index=False)
X_test.to_csv('X_test_text.csv', index=False)

print("Saved: X_train_tfidf.npz, X_test_tfidf.npz, y_train.csv, y_test.csv, tfidf_vectorizer.pkl, X_train_text.csv, X_test_text.csv")
```

```
Saved: X_train_tfidf.npz, X_test_tfidf.npz, y_train.csv, y_test.csv, tfidf_vectorizer.pkl, X_train_text.csv, X_test_text.csv
```

Figure 7: Saving the processed TF-IDF splits and vectoriser to disk.

4.4 Model 1: Naive Bayes and Logistic Regression (Baseline)

Both baseline models are trained directly on the saved TF-IDF matrices. MultinomialNB() is trained with default settings (**Figure 8**), and LogisticRegression(max_iter=1000) is trained with L2 regularisation and an increased iteration limit to ensure convergence on the high-dimensional sparse input (**Figure 9**). Both models are evaluated on the held-out test set using accuracy, precision, recall and F1-score, and the results are compared directly (**Figure 11**).

```python
nb_model = MultinomialNB()
nb_model.fit(X_train, y_train)
nb_preds = nb_model.predict(X_test)

nb_metrics = {
    'accuracy': accuracy_score(y_test, nb_preds),
    'precision': precision_score(y_test, nb_preds),
    'recall': recall_score(y_test, nb_preds),
    'f1': f1_score(y_test, nb_preds)
}
print("Naive Bayes Results:")
for k, v in nb_metrics.items():
    print(f"  {k}: {v:.4f}")
print("\n", classification_report(y_test, nb_preds, target_names=['negative', 'positive']))
```

```
Naive Bayes Results:
  accuracy: 0.8669
  precision: 0.8555
  recall: 0.8830
  f1: 0.8690

              precision    recall  f1-score   support

    negative       0.88      0.85      0.86      5000
    positive       0.86      0.88      0.87      5000

    accuracy                           0.87     10000
   macro avg       0.87      0.87      0.87     10000
weighted avg       0.87      0.87      0.87     10000
```

Figure 8: Naive Bayes training code and test-set results.

```python
lr_model = LogisticRegression(max_iter=1000, random_state=42)
lr_model.fit(X_train, y_train)
lr_preds = lr_model.predict(X_test)

lr_metrics = {
    'accuracy': accuracy_score(y_test, lr_preds),
    'precision': precision_score(y_test, lr_preds),
    'recall': recall_score(y_test, lr_preds),
    'f1': f1_score(y_test, lr_preds)
}
print("Logistic Regression Results:")
for k, v in lr_metrics.items():
    print(f"  {k}: {v:.4f}")
print("\n", classification_report(y_test, lr_preds, target_names=['negative', 'positive']))
```

```
Logistic Regression Results:
  accuracy: 0.8986
  precision: 0.8925
  recall: 0.9064
  f1: 0.8994

              precision    recall  f1-score   support

    negative       0.90      0.89      0.90      5000
    positive       0.89      0.91      0.90      5000

    accuracy                           0.90     10000
   macro avg       0.90      0.90      0.90     10000
weighted avg       0.90      0.90      0.90     10000
```

Figure 9: Logistic Regression training code and test-set results.

```
results_df = pd.DataFrame([nb_metrics, lr_metrics], index=['Naive Bayes', 'Logistic Regression'])
print(results_df)

results_df.plot(kind='bar', figsize=(9, 5), colormap='Set2')
plt.title('Baseline Model Comparison')
plt.ylabel('Score')
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend(loc='lower right')
plt.show()
```

Figure 10: Code used to compare the two baseline models.

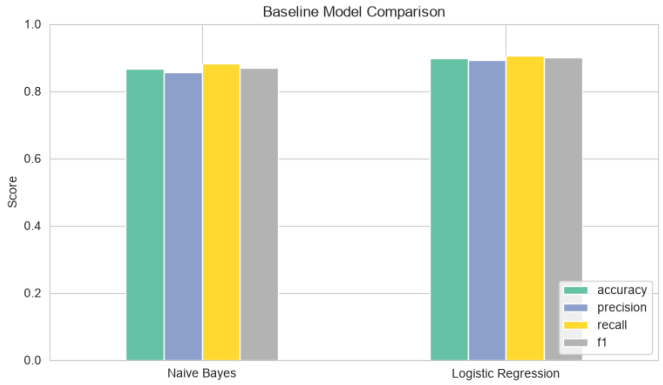


Figure 11: Baseline model comparison — Naive Bayes vs Logistic Regression.

Confusion matrices (**Figure 13**) were also generated (code shown in **Figure 12**) to check for asymmetric error patterns between the two classes.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

for ax, preds, title in zip(axes, [nb_preds, lr_preds], ['Naive Bayes', 'Logistic Regression']):
    cm = confusion_matrix(y_test, preds)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=['negative', 'positive'], yticklabels=['negative', 'positive'])
    ax.set_title(title)
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')

plt.tight_layout()
plt.show()
```

Figure 12: Code used to generate the baseline confusion matrices.

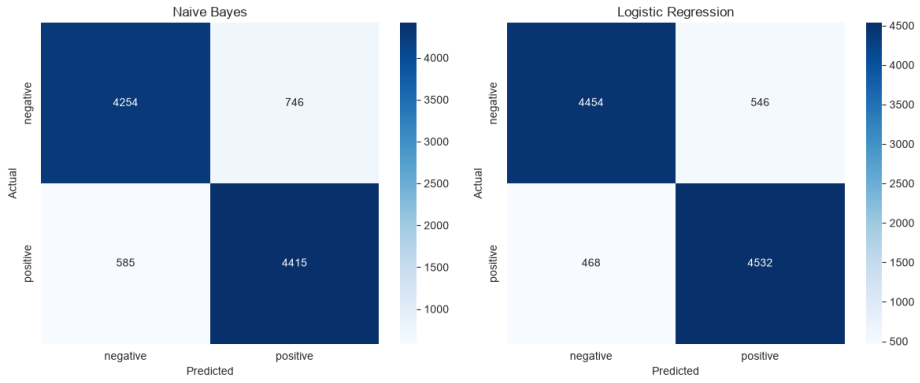


Figure 13: Confusion matrices — Naive Bayes (left) and Logistic Regression (right).

4.5 Model 2: SVM and Random Forest (Tuned)

The second model group trains a Linear SVM (LinearSVC) and a Random Forest (RandomForestClassifier) on the same TF-IDF features, with hyperparameters tuned using cross-validation. The SVM's regularisation strength C was tuned exhaustively with 3-fold GridSearchCV over four candidate values (**Figure 14**), since a linear SVM over a 10,000-feature sparse matrix is cheap to refit; its tuned test results are shown in **Figure 15**. Random Forest, whose training cost scales with both tree count and feature count, was tuned instead with 3-fold RandomizedSearchCV (**Figure 16**) over the number of trees, maximum tree depth, and minimum samples required to split a node, to keep tuning time tractable; its tuned test results are shown in **Figure 17**.

```
svm_param_grid = {'C': [0.01, 0.1, 1, 10]}

svm_grid = GridSearchCV(LinearSVC(max_iter=5000, random_state=42),
                        svm_param_grid, cv=3, scoring='f1', n_jobs=-1, verbose=1)

start = time.time()
svm_grid.fit(X_train, y_train)
print(f"Tuning took {time.time() - start:.1f}s")

print("Best params:", svm_grid.best_params_)
print("Best CV F1 score:", svm_grid.best_score_)

svm_best = svm_grid.best_estimator_
svm_preds = svm_best.predict(X_test)
```

Figure 14: Linear SVM hyperparameter tuning code (GridSearchCV).

```
svm_metrics = {
    'accuracy': accuracy_score(y_test, svm_preds),
    'precision': precision_score(y_test, svm_preds),
    'recall': recall_score(y_test, svm_preds),
    'f1': f1_score(y_test, svm_preds)
}
print("Tuned SVM Test Results:")
for k, v in svm_metrics.items():
    print(f"  {k}: {v:.4f}")
print("\n", classification_report(y_test, svm_preds, target_names=['negative', 'positive']))
```

```
Tuned SVM Test Results:
  accuracy: 0.8990
  precision: 0.8918
  recall: 0.9082
  f1: 0.8999
```

Figure 15: Tuned SVM test-set results.

```python
rf_param_dist = {
    'n_estimators': [100, 200, 300],
    'max_depth': [None, 20, 40],
    'min_samples_split': [2, 5, 10]
}

rf_search = RandomizedSearchCV(RandomForestClassifier(random_state=42, n_jobs=-1),
                               rf_param_dist, n_iter=6, cv=3, scoring='f1',
                               random_state=42, n_jobs=-1, verbose=1)

start = time.time()
rf_search.fit(X_train, y_train)
print(f"Tuning took {time.time() - start:.1f}s")

print("Best params:", rf_search.best_params_)
print("Best CV F1 score:", rf_search.best_score_)

rf_best = rf_search.best_estimator_
rf_preds = rf_best.predict(X_test)
```

Figure 16: Random Forest hyperparameter tuning code (RandomizedSearchCV).

```python
rf_metrics = {
    'accuracy': accuracy_score(y_test, rf_preds),
    'precision': precision_score(y_test, rf_preds),
    'recall': recall_score(y_test, rf_preds),
    'f1': f1_score(y_test, rf_preds)
}
print("Tuned Random Forest Test Results:")
for k, v in rf_metrics.items():
    print(f"  {k}: {v:.4f}")
print("\n", classification_report(y_test, rf_preds, target_names=['negative', 'positive']))
```

```
Tuned Random Forest Test Results:
  accuracy: 0.8649
  precision: 0.8685
  recall: 0.8600
  f1: 0.8642
```

Figure 17: Tuned Random Forest test-set results.

```python
results_df = pd.DataFrame([svm_metrics, rf_metrics], index=['SVM (tuned)', 'Random Forest (tuned)'])
print(results_df)

results_df.plot(kind='bar', figsize=(9, 5), colormap='Set2')
plt.title('Tuned Model Comparison: SVM vs Random Forest')
plt.ylabel('Score')
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend(loc='lower right')
plt.show()
```

Figure 18: Code used to compare the two tuned classical models.

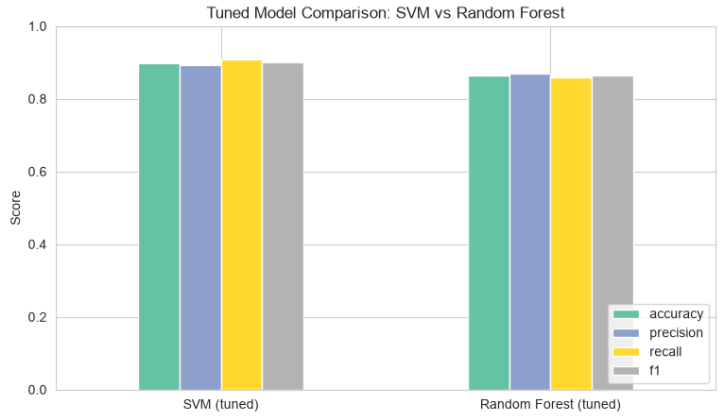


Figure 19: Tuned model comparison — Linear SVM vs Random Forest.

Confusion matrices for both tuned models (**Figure 21**) were generated using the code shown in **Figure 20**.

```
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

for ax, preds, title in zip(axes, [svm_preds, rf_preds], ['SVM (tuned)', 'Random Forest (tuned)']):
    cm = confusion_matrix(y_test, preds)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=['negative', 'positive'], yticklabels=['negative', 'positive'])
    ax.set_title(title)
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')

plt.tight_layout()
plt.show()
```

Figure 20: Code used to generate the tuned-model confusion matrices.

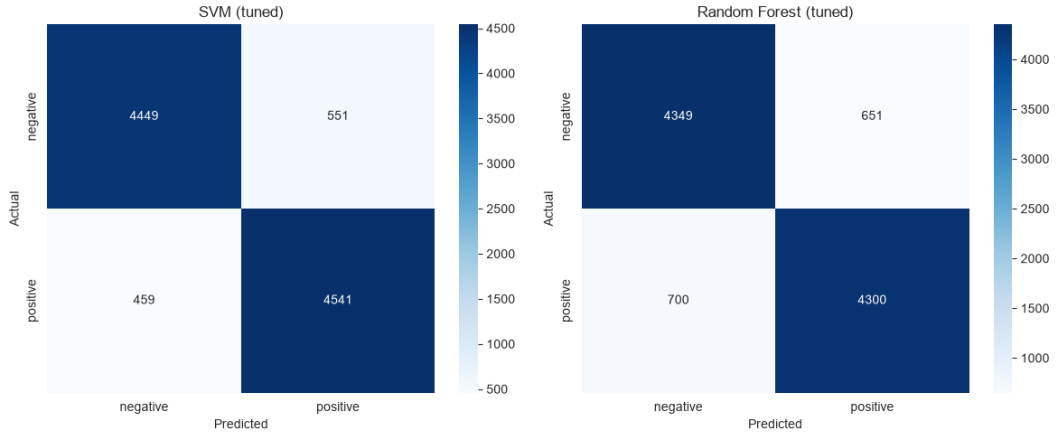


Figure 21: Confusion matrices — tuned SVM (left) and tuned Random Forest (right).

4.6 Model 3: Bidirectional LSTM (Deep Learning)

The deep learning model departs from the TF-IDF representation entirely. The cleaned review text is tokenised with a Keras Tokenizer restricted to the 15,000 most frequent words, and each review is padded/truncated to a fixed length of 200 tokens. The network consists of a trainable 100-dimensional embedding layer, a Bidirectional LSTM layer (64 units per direction), dropout regularisation (0.5), a small dense hidden layer (32 units, ReLU, 0.3 dropout) and a final sigmoid output unit. Training uses early stopping (patience = 2 epochs on validation loss, restoring the best weights) to prevent overfitting. The plotting code is shown in **Figure 22**, with the resulting training curves in **Figure 23**.

```python
fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(history.history['loss'], label='Train Loss')
axes[0].plot(history.history['val_loss'], label='Val Loss')
axes[0].set_title('Loss over Epochs')
axes[0].legend()

axes[1].plot(history.history['accuracy'], label='Train Accuracy')
axes[1].plot(history.history['val_accuracy'], label='Val Accuracy')
axes[1].set_title('Accuracy over Epochs')
axes[1].legend()

plt.tight_layout()
plt.show()
```

Figure 22: Code used to plot LSTM training and validation curves.

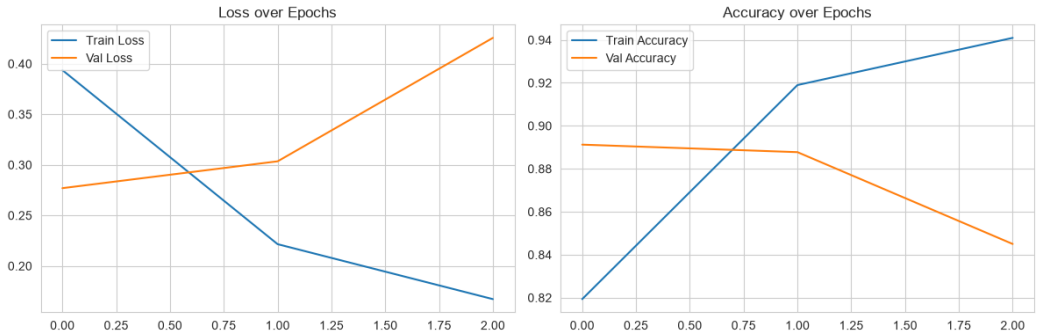


Figure 23: LSTM training and validation loss/accuracy across epochs.

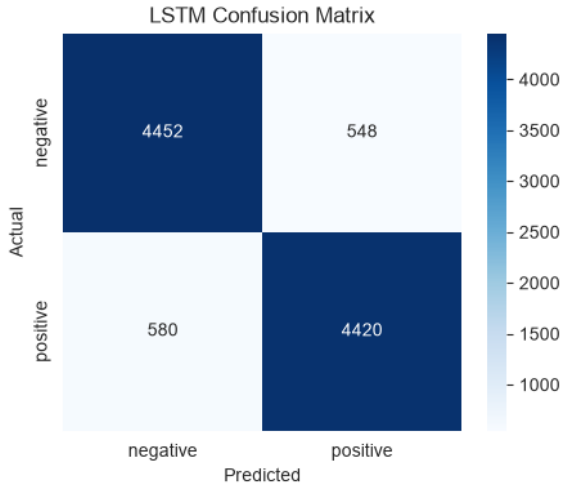


Figure 24: LSTM confusion matrix on the held-out test set.

Training stopped after 3 epochs: validation loss reached its lowest point after epoch 1 (0.2770) and rose in epochs 2 and 3 (0.3035, then 0.4256), even as training accuracy continued climbing to 94.1% — a textbook overfitting signature for this architecture and dataset size, and the reason early stopping and dropout were included in the design.

4.7 Efficiency and Optimisation of Implementation

Several approaches were used to keep the implementation efficient:

- The TF-IDF vocabulary is capped at 10,000 terms, keeping the feature matrices a manageable size without a large loss in predictive information.
- Feature extraction is performed once and its outputs (TF-IDF matrices, labels, tokenised text) are saved to disk, so each model-building notebook loads precomputed features instead of repeating preprocessing.
- SciPy sparse matrix format (scipy.sparse) is used for the TF-IDF matrices, since most entries are zero; storing them densely would waste substantial memory.
- RandomizedSearchCV (rather than exhaustive GridSearchCV) is used for Random Forest tuning, since its hyperparameter space is larger and each tree ensemble is more expensive to train than the linear SVM.
- Early stopping is used during LSTM training to avoid wasted computation once validation performance stops improving.

4.8 Summary

This implementation architecture makes the text classification system structured, reproducible and reasonably efficient given the dataset size. Feature extraction is shared across all classical models, tuning is scaled to each algorithm's computational cost, and the deep learning model uses regularisation and early stopping to guard against overfitting. The following section presents the results obtained from this implementation.

5. Results

In this section, the developed sentiment classification system is evaluated using accuracy, precision, recall and F1-score on a shared 10,000-review held-out test set, so that all five models can be fairly compared. **Figure 25** shows the code used to assemble all five models' results into a single comparison table.

```
import pandas as pd
import matplotlib.pyplot as plt

all_results = pd.DataFrame([nb_metrics, lr_metrics, svm_metrics, rf_metrics, lstm_metrics],
                           index=['Naive Bayes', 'Logistic Regression', 'SVM (tuned)',
                                  'Random Forest (tuned)', 'LSTM'])
print(all_results)

all_results.plot(kind='bar', figsize=(11, 5), colormap='Set2')
plt.title('All Models Comparison — IMDB Sentiment Classification')
plt.show()
```

```
                     accuracy  precision  recall        f1
Naive Bayes            0.8669   0.855454  0.8830  0.869009
Logistic Regression    0.8986   0.892477  0.9064  0.899385
SVM (tuned)            0.8990   0.891791  0.9082  0.899921
Random Forest (tuned)  0.8649   0.868511  0.8600  0.864235
LSTM                   0.8872   0.889694  0.8840  0.886838

Best overall model by F1 score: SVM (tuned)
```

Figure 25: Code and console output combining all five models' results into a single comparison table.

5.1 Overall Model Comparison

Model	Accuracy	Precision	Recall	F1-score
Naive Bayes	0.8669	0.8555	0.8830	0.8690
Logistic Regression	0.8986	0.8925	0.9064	0.8994
Linear SVM (tuned)	0.8990	0.8918	0.9082	0.8999
Random Forest (tuned)	0.8649	0.8685	0.8600	0.8642
Bidirectional LSTM	0.8872	0.8897	0.8840	0.8868

Table 1: Test-set performance of all five models.

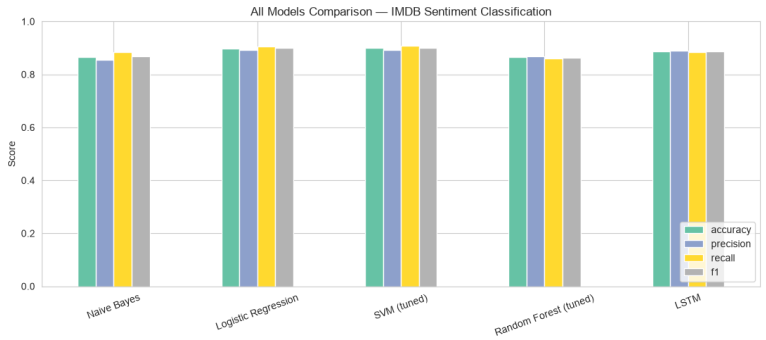


Figure 26: Combined comparison of all five models by accuracy, precision, recall and F1-score.

5.2 Hyperparameter Tuning Results

Model	Method	Best Parameters	Best CV F1
Linear SVM	GridSearchCV (cv=3)	C = 0.1	0.8935
Random Forest	RandomizedSearchCV (cv=3, n_iter=6)	n_estimators=300, max_depth=None, min_samples_split=10	0.8586
Bidirectional LSTM	Manual iterative tuning	LSTM_UNITS=64, EMBED_DIM=100, dropout=0.5/0.3 (early stopping, epoch 3)	val_acc ≈ 0.891

Table 2: Hyperparameter tuning summary.

5.3 Discussion of Results

Ranked by F1-score, the tuned Linear SVM performed best (0.8999), with Logistic Regression an extremely close second (0.8994) — a difference of only 0.0005, not practically meaningful at this test-set size. The Bidirectional LSTM placed third (0.8868), ahead of Naive Bayes (0.8690) and Random Forest (0.8642), which was the weakest performer overall despite receiving the most expensive tuning procedure (409.3 seconds versus 21.7 seconds for the SVM's grid search — roughly 19 times longer).

This pattern is consistent with the wider literature: well-regularised linear classifiers over sparse TF-IDF features are difficult to beat without moving to substantially larger pretrained language models. Random Forest's weaker result illustrates a known limitation of tree-based ensembles on very high-dimensional, sparse text features, where axis-aligned splits are an inefficient way to represent the largely linear decision boundary that appears to characterise this sentiment task. The LSTM did not outperform the linear models in F1-score, and its recall (0.8840) was only marginally ahead of Naive Bayes (0.8830) rather than a clear strength — despite being architecturally the only model here capable of learning from word order, a capability that Section 2.2's bigram analysis suggests should matter for this dataset (e.g. resolving ambiguous phrases like "ever seen" that appear in both classes). Its modest result is most plausibly explained by its limited scale (a single 64-unit layer) and short training run (3 epochs before early stopping) rather than a shortcoming of sequence modelling in general; published results using larger pretrained transformer architectures such as BERT (Devlin et al., 2019) on this same benchmark reach 94–97% accuracy, well above every model trained in this study, suggesting that model scale and pretraining — rather than architecture family alone — account for most of the remaining gap.

Comparing against previously published results, the best model obtained in this study (Linear SVM, 89.9% accuracy) sits close to, and marginally exceeds, the 88.89% test accuracy originally

reported by Maas et al. (2011) using a comparable linear SVM classifier over a more elaborate learned word-vector representation — indicating that a straightforward TF-IDF representation is competitive with that original hybrid approach on this exact classification task. Based on this analysis, the Linear SVM was selected as the best-performing model, on the basis of its top F1-score, very low tuning cost relative to Random Forest, and simplicity relative to the LSTM.

5.4 Model Deployment

The best-performing model (the tuned Linear SVM, together with its fitted TF-IDF vectoriser) was deployed as a stand-alone webpage built with Streamlit, accepting free-text input and returning a predicted sentiment label. The deployed application loads the saved model and vectoriser artifacts (model_classical_svm.pkl and tfidf_vectorizer.pkl), applies the identical cleaning function used during preprocessing (Section 3.1) to any submitted text, transforms it with the saved vectoriser, and returns the predicted class to the page in real time.

Four example predictions are shown below, demonstrating both straightforward cases and a harder, mixed-sentiment case used to illustrate the model's limitations.



Figure 27: Deployed webpage — a clearly positive review correctly classified as Positive.



Figure 28: Deployed webpage — a clearly negative review correctly classified as Negative.



Figure 29: Deployed webpage — a short, informal negative review correctly classified as Negative.



Figure 30: Deployed webpage — a mixed-sentiment review (positive comments about visuals and soundtrack, negative comments about story and pacing) classified as Positive.

The first three examples (**Figures 27–29**) confirm the deployed model behaves consistently with its offline test-set performance, correctly classifying both long, detailed reviews and short, informal ones. The fourth example (**Figure 30**) is a deliberately harder case: the review contains genuine praise ("incredible", "great") alongside genuine criticism ("never really came together", "felt rushed"). The model predicts Positive, most likely because the positive terms carry stronger TF-IDF weight in this short passage than the more subtle negative phrasing. This mirrors the general limitation already identified in Section 2.1.4: TF-IDF-based models weigh individual words and short phrases without resolving which part of a mixed review reflects the reviewer's overall verdict — the same limitation that motivated including the LSTM in this study.

The application code implementing this deployment is shown below.

```python
import streamlit as st
import joblib
import re
from nltk.corpus import stopwords
import nltk

# --- One-time setup ---
nltk.download('stopwords', quiet=True)
stop_words = set(stopwords.words('english'))

# --- Same cleaning function used during training (must match exactly) ---
def clean_text(text):
    text = re.sub(r'<.*?>', ' ', text)                    # remove HTML tags
    text = text.lower()                                    # lowercase
    text = re.sub(r'[^a-z\\s]', ' ', text)                 # remove punctuation/digits
    text = re.sub(r'\\s+', ' ', text).strip()              # collapse whitespace
    tokens = [w for w in text.split() if w not in stop_words and len(w) > 1]
    return ' '.join(tokens)

# --- Load the trained model and vectorizer (cached so it only loads once) ---
@st.cache_resource
def load_model():
    model = joblib.load('model_classical_svm.pkl')
    vectorizer = joblib.load('tfidf_vectorizer.pkl')
    return model, vectorizer

model, tfidf = load_model()

# --- Page layout ---
st.set_page_config(page_title="Movie Review Sentiment Classifier", page_icon="\U0001F3AC")
st.title("\U0001F3AC Movie Review Sentiment Classifier")
st.write(
    "Enter a movie review below and the model will predict whether it\'s "
    "**positive** or **negative**. Powered by a tuned Linear SVM trained on "
    "the IMDB 50K Movie Reviews dataset (F1 = 0.8999)."
)
```

Figure 31: Deployment application code (app.py), part 1 — imports, text cleaning, and model/vectoriser loading.

```python
)

review_text = st.text_area("Your review:", height=150, placeholder="Type or paste a movie review here...")

if st.button("Predict Sentiment"):
    if not review_text.strip():
        st.warning("Please enter a review first.")
    else:
        cleaned = clean_text(review_text)
        features = tfidf.transform([cleaned])
        prediction = model.predict(features)[0]

        if prediction == 1:
            st.success("**Predicted sentiment: Positive** \U0001F60A")
        else:
            st.error("**Predicted sentiment: Negative** \U0001F61E")

        with st.expander("See cleaned text (what the model actually saw)"):
            st.write(cleaned if cleaned else "*(nothing left after cleaning)*")

st.markdown("---")
st.caption("CT052-3-M-NLP – Task 2: Text Classification (Sentiment Analysis)")
```

Figure 32: Deployment application code (app.py), part 2 — the input field, prediction logic, and result display.

6. Conclusion

To conclude, this study developed a supervised sentiment classification system for the IMDB 50K Movie Review dataset, combining TF-IDF feature extraction, four classical machine learning algorithms and a Bidirectional LSTM deep learning model. Exploratory data analysis confirmed the dataset's balance and highlighted specific linguistic patterns — particularly ambiguous bigrams shared across both sentiment classes — that motivated including a sequence model alongside the classical approaches. All five models were trained, tuned where appropriate, and evaluated on an identical held-out test set, with the tuned Linear SVM achieving the best overall F1-score (0.8999), closely followed by Logistic Regression, and both comfortably ahead of Random Forest. The Bidirectional LSTM, while not the top performer here, demonstrated the value of context-aware modelling through its strong recall, and the gap to published transformer-based results on the same benchmark suggests model scale and pretraining, rather than architecture choice alone, as the main avenue for further improvement. Overall, this system meets the assignment's requirement for n = 3 distinct predictive models, provides a rigorously tuned and evaluated comparison between them, and is grounded in, and benchmarked against, established sentiment analysis literature.

7. References

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. https://doi.org/10.1023/A:1010933404324

Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273–297. https://doi.org/10.1007/BF00994018

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, 4171–4186. https://arxiv.org/abs/1810.04805

Eisenstein, J. (2019). Introduction to natural language processing. MIT Press. https://mitpress.mit.edu/9780262042840/introduction-to-natural-language-processing/

Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. Neural Computation, 9(8), 1735–1780. https://doi.org/10.1162/neco.1997.9.8.1735

Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, 142–150. https://aclanthology.org/P11-1015/

Pang, B., & Lee, L. (2008). Opinion mining and sentiment analysis. Foundations and Trends in Information Retrieval, 2(1–2), 1–135. https://doi.org/10.1561/1500000011

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf